How to Find Time Complexity

DSA lecture notes, part 3 (from the YouTube lecture, course “Complete DSA from A to Z”). The lecture is in Hindi/Hinglish; these notes are in simple English. Code examples marked “my example” are added by me to make the idea clear.

The whole lecture in 30 seconds

- Knowing complexity makes you write **faster programs**. With small input you will not notice, but with a million records a slow program becomes a big problem.
- To find the complexity, watch two things: **how the input size n grows**, and **how many times your statements run** when n grows.
- Three steps: **1) find the loops and recursive calls, 2) count how many times things run in terms of n, 3) drop constants and the smaller terms.**

1. The story of Ram and Shyam

Two people get the **same problem** in an interview. Both are good programmers (Shyam is even a bit better at coding). The only difference: **Ram knows DSA and complexity, Shyam does not.**

	Ram (knows complexity)	Shyam (does not)
How he thinks	Before writing a loop inside a loop, he thinks 10 times: “this will increase my complexity, is there a better way?”	Only thinks “how do I solve it?” and happily writes 1, 2 or 3 nested loops.
Sorting example	Picks a faster sort such as **merge sort** or **quick sort**.	Uses simple **bubble sort**.
With 10 elements	Works fine.	Works fine. You cannot see any difference.
With 1 million records	Still fast.	Becomes very slow.

Lesson: a program that “works” is not enough. As the input grows, the complexity of your logic decides whether the program stays fast. Keep it in mind in **interviews, GATE and competitive programming**, where running time is checked.

2. Quick reminder of the common complexities

These were covered in the previous video (watch it if any of this is new).

Complexity	Simple meaning
O(1) constant	Number of steps does not change with input. Example: 4 statements always run 4 times, whether n is 10 or 1 million.
O(log n) logarithmic	Input is **divided** at every step, like binary search.
O(n) linear	If input doubles, steps double.
O(n^2) quadratic	Steps grow with the square. If n goes 1 → 2, steps go 1 → 4.
O(2^n) exponential	Steps double with every extra input.
O(n!) factorial	You try every arrangement (permutation), for example.

3. The golden rule: watch two things

- **Input size**: how is n growing?
- **Statements executed**: when n grows, how many times do your statements run?

Find the **relation between the input and the number of statements run**. That relation is your complexity.

Important: a loop is not automatically O(n). A loop only counts if it **depends on the input size**.

```
// my example: loop does NOT depend on n -> O(1)
for (int i = 1; i <= 100; i++) { ... }   // always 100 times, even if n = 1 million

// my example: loop depends on n -> O(n)
for (int i = 1; i <= n; i++) { ... }     // n = 10 -> 10 times, n = 5 -> 5 times
```

4. Three steps to find complexity

Step 1: Identify the loops and recursive calls

Break your code into small pieces and look at **where loops run and where recursive calls happen**.

Step 2: Count how many times they run

What you see	Complexity
Only plain statements (no loops that depend on n)	**O(1)**, constant
One loop running from 1 to n	**O(n)**
Loop inside a loop, both depending on n	**$O(n^2)$**
Loop inside loop inside loop	**$O(n^3)$**
m nested loops	**$O(n^m)$**
Recursive calls	Depends on how **deep** the recursion goes (see below)

Why do nested loops multiply?

```
for (int i = 1; i <= n; i++) {        // outer loop: n times
    for (int j = 1; j <= n; j++) {    // inner loop: n times for EACH i
        // O(1) work
    }
}
```

When i = 1 the inner loop runs n times. When i = 2 it runs n times again. And so on, n times in total. So the total is **$n \times n = n^2$**. Add one more loop inside and it becomes $n \times n \times n = n^3$.

> Be careful with nested loops. As n grows, every extra loop makes the program much slower. Use them only when you really cannot avoid them.

Recursion

For recursive code, the complexity depends on **how deep the recursive calls go**. You may need to write a **recurrence relation** and solve it with the **Master theorem** or the **substitution method**. (The lecture only mentions this; it is taught in detail later.)

Step 3: Drop constants and non-dominant terms

This is the **easiest step**. Two simple rules:

- **Drop constants.** A fixed number does not grow with the input, so it does not matter.
- **Drop non-dominant terms.** If one term is far bigger than the others as n grows, keep only the biggest one. Think of 100 million + 100: the 100 is so tiny that we ignore it.

What you counted	Answer in Big O	Why
$n + 100$	**O(n)**	100 is a constant, drop it
$n^2 + n$	**O(n^2)**	n is small compared to n^2
$n^3 + n^2 + n + 10$	**O(n^3)**	Keep only the biggest term, n^3

> **Remember:** in Big O always keep the **biggest term** and throw away the rest.

5. Practice and quick check

The lecture ends by saying that the real understanding comes from **practice**: in the next video he solves around 10 examples to find complexities. Here are a few of my own to try now (answers below):

```
A)  int a = 5; int b = 10; int c = a + b;

B)  for (int i = 1; i <= n; i++) { ... }

C)  for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++) { ... }

D)  one loop to n, then another separate loop to n (not nested)

E)  total steps = n*n*n + n*n + n + 10
```

Answers: A) O(1) B) O(n) C) O(n^2) D) O(n) (n + n = 2n, drop the constant 2) E) O(n^3).

Also remember to **take attendance in the comment section** and like the video, as the teacher asks at the end.